Rockchip Linux CMA Developer Guide

ID: RK-KF-YF-117

Release Version: V1.1.0

Release Date: 2022-06-02

Security Level: □Top-Secret □Secret □Internal ■Public

DISCLAIMER

THIS DOCUMENT IS PROVIDED “AS IS”. ROCKCHIP ELECTRONICS CO., LTD.(“ROCKCHIP”)DOES NOT PROVIDE ANY WARRANTY OF ANY KIND, EXPRESSED, IMPLIED OR OTHERWISE, WITH RESPECT TO THE ACCURACY, RELIABILITY, COMPLETENESS,MERCHANTABILITY, FITNESS FOR ANY PARTICULAR PURPOSE OR NON-INFRINGEMENT OF ANY REPRESENTATION, INFORMATION AND CONTENT IN THIS DOCUMENT. THIS DOCUMENT IS FOR REFERENCE ONLY. THIS DOCUMENT MAY BE UPDATED OR CHANGED WITHOUT ANY NOTICE AT ANY TIME DUE TO THE UPGRADES OF THE PRODUCT OR ANY OTHER REASONS.

Trademark Statement

"Rockchip", "瑞芯微", "瑞芯" shall be Rockchip’s registered trademarks and owned by Rockchip. All the other trademarks or registered trademarks mentioned in this document shall be owned by their respective owners.

All rights reserved. ©2022. Rockchip Electronics Co., Ltd.

Beyond the scope of fair use, neither any entity nor individual shall extract, copy, or distribute this document in any form in whole or in part without the written approval of Rockchip.

Rockchip Electronics Co., Ltd.

No.18 Building, A District, No.89, software Boulevard Fuzhou, Fujian,PRC

Website: www.rock-chips.com

Customer service Tel: +86-4007-700-590

Customer service Fax: +86-591-83951833

Customer service e-Mail: fae@rock-chips.com

Preface

Overview

This is a Rockchip platform Linux CMA (Contiguous Memory Allocator) developer guide for chips like RK3588 and RK3568. The document covers the CMA driver framework, memory reservation allocation methods, and debugging techniques to help developers solve memory fragmentation issues for peripherals requiring large contiguous physical memory blocks.

Product Version

Chip Models	Kernel Version
RK3588, RK3568, RK3399, RK3288, RK3368, RV1126, PX30	Linux-4.4, Linux-4.19, Linux-5.10

Intended Audience

This document (this guide) is primarily intended for the following engineers:

Technical Support Engineers

Software Development Engineers

Revision History

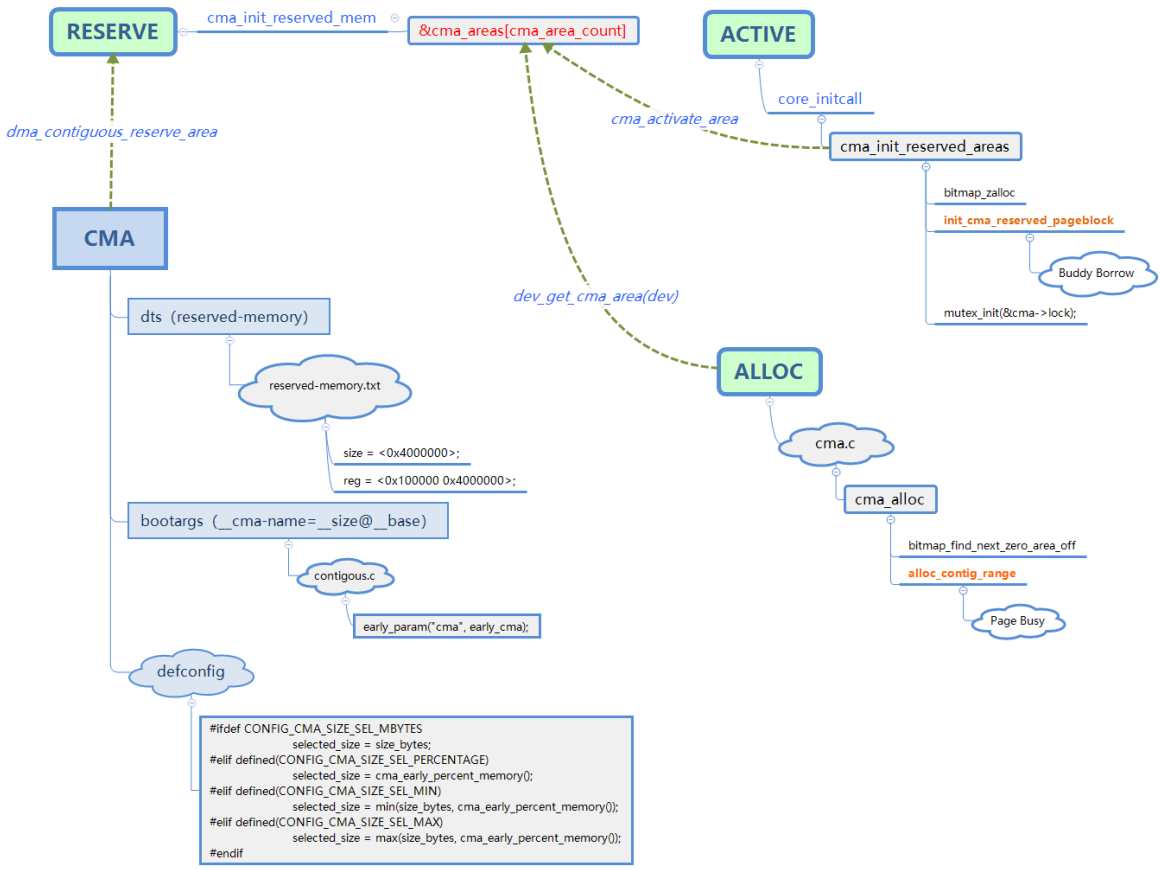
Version	Author	Date	Description
V1.0.0	Xu Jianqun	2020-10-18	Initial version
V1.1.0	Xu Jianqun	2022-06-02	Added CMA framework diagram

Table of Contents

Rockchip Linux CMA Developer Guide

1. CMA Driver Framework
2. Memory Reservation Allocation Methods
   - 2.1 Allocation Method 1: Reserve 16 MByte via menuconfig
   - 2.2 Allocation Method 2: Via the reserved-memory node in DTS
3. Methods for Allocating Exclusive Memory
   - 3.1 Allocation Method 1: Independently Existing Memory Blocks Subject to System Borrowing
   - 3.2 Allocation Method 2: Allocating Memory Blocks via memblock
4. Memory Reservation Debugging Methods
   - 4.1 CMA Debug Macro Switches
   - 4.2 CMA Debug Nodes
5. Memory Reservation Information During Boot

1. CMA Driver Framework



CMA primarily consists of three driver components:

1. size@address

There are three methods to pass parameters, with the priority order being DTS > CMDLINE > DEFCONFIG. Specifically, if the DTS contains a valid node, the DTS node configuration will be used as the final parameter. Otherwise, the system checks whether the cmdline includes cma=size@base. Finally, it checks whether the DEFCONFIG contains the four macros mentioned above.

2. reserved cma_areas[]

Multiple CMA nodes can be configured via DTS, whereas only the default node can be defined via cmdline and DEFCONFIG. All reserved memory is stored in the global cma_areas[] array.

3. active cma_area

Parameter parsing and memory reservation are completed during the kernel's setup_arch phase. The active phase is invoked via core_initcall to ensure completion before any Driver initialization. The active phase primarily handles bitmap creation for the CMA area and adds CMA pages to the buddy system.

2. Memory Reservation Allocation Methods

2.1 Allocation Method 1: Reserve 16 MByte via menuconfig

CONFIG_DMA_CMA=y
CONFIG_CMA_SIZE_MBYTES=16
CONFIG_CMA_SIZE_SEL_MBYTES=y

2.2 Allocation Method 2: Via the reserved-memory node in DTS

Refer to the technical document `Documentation/devicetree/bindings/reserved-memory/reserved-memory.txt`.

```
reserved-memory {
    #address-cells = <1>;
    #size-cells = <1>;
    ranges;

    /* global autoconfigured region for contiguous allocations */
    linux,cma {
        compatible = "shared-dma-pool";
        reusable;
        size = <0x1000000>;
        alignment = <0x2000>;
        linux,cma-default;
    };
};
```

3. Methods for Allocating Exclusive Memory

The methods described above allocate system shared memory blocks, which can lead to fragmentation after multiple allocation and release cycles across various devices. Certain peripherals require large contiguous CMA memory blocks; fragmentation can cause these peripherals to malfunction. Therefore, it is necessary to allocate dedicated memory blocks for specific devices.

3.1 Allocation Method 1: Independently Existing Memory Blocks Subject to System Borrowing

```
reserved-memory {
    #address-cells = <1>;
    #size-cells = <1>;
    ranges;

    cma_region: region@88000000 {
        compatible = "shared-dma-pool";
        reusable;
        reg = <0x88000000 0x1800000>;
    };
};
```

The memory block allocated by this method can be assigned to a specific device using `of_reserved_mem_device_init`.

```
#include <linux/of_reserved_mem.h>
static inline int of_reserved_mem_device_init(struct device *dev)
{
    return of_reserved_mem_device_init_by_idx(dev, dev->of_node, 0);
}
```

3.2 Allocation Method 2: Allocating Memory Blocks via memblock

```
reserved-memory {
    #address-cells = <1>;
    #size-cells = <1>;
    ranges;

    display_reserved: framebuffer@78000000 {
        reg = <0x78000000 0x800000>;
    };
};
```

You can view the allocation results via the `/sys/kernel/debug/memblock/reserved` node.

The memblock ranges displayed by this node are merged and organized.

Memory blocks allocated using this method are not managed by the kernel page allocator; users must create page scale lists according to their specific requirements.

```
console:/sys/kernel/debug/cma # cat /sys/kernel/debug/memblock/reserved
    0: 0x60004000.0x60007fff
    1: 0x60100000.0x61404a07
    2: 0x62e00000.0x62eeffff
    3: 0x68300000.0x6830d07f
...
```

4. Memory Reservation Debugging Methods

4.1 CMA Debug Macro Switches

```
CONFIG_CMA_DEBUGFS=y # Enable the debug file node
CONFIG_CMA_DEBUG=y # Print CMA Log Information
```

4.2 CMA Debug Nodes

```
console:/sys/kernel/debug/cma # ls
cma-region@8800 cma-reserved
console:/sys/kernel/debug/cma/cma-reserved # ls
alloc    bitmap free     order_per_bit      used
base_pfn count  maxchunk
```

- alloc: Allocates memory from the current CMA block, in pages.
- bitmap: Marks whether each page in the current CMA block is allocated; 1 indicates allocated, 0 indicates unallocated.
- free: Releases memory from the current CMA block, in pages.
- order_per_bit: Represents the page order represented by a single bit.
- used: The number of currently allocated pages, in pages.
- base_pfn: The page frame number of the first page in the current CMA block.
- maxchunk: The maximum number of contiguous free pages in the current CMA block, in pages.

5. Memory Reservation Information During Boot

```
[    0.000000] Booting Linux on physical CPU 0xf00
[    0.000000] CPU: ARMv7 Processor [410fc075] revision 5 (ARMv7), cr=10c5387d
[    0.000000] CPU: div instructions available: patching division code
[    0.000000] CPU: PIPT / VIPT nonaliasing data cache, VIPT aliasing
instruction cache
[    0.000000] OF: fdt: Machine model: Rockchip RK3126 bnd-d708 board
[    0.000000] earlycon: uart8250 at MMIO32 0x20068000 (options '')
[    0.000000] bootconsole [uart8250] enabled
[    0.000000] Memory policy: Data cache writealloc
[    0.000000] Reserved memory: created CMA memory pool at 0x88000000, size 24
MiB
[    0.000000] OF: reserved mem: initialized node region@88000000, compatible id
shared-dma-pool
[    0.000000] cma: Reserved 16 MiB at 0x9f000000
[    0.000000] On node 0 totalpages: 258560
[    0.000000] Normal zone: 1728 pages used for memmap
[    0.000000] Normal zone: 0 pages reserved
[    0.000000] Normal zone: 193024 pages, LIFO batch:63
[    0.000000] HighMem zone: 65536 pages, LIFO batch:15
[    0.000000] psci: probing for conduit method from DT.
[    0.000000] psci: PSCIv65535.65535 detected in firmware.
[    0.000000] psci: Using standard PSCI v0.2 function IDs
[    0.000000] psci: MIGRATE_INFO_TYPE not supported.
[    0.000000] psci: SMC Calling Convention v1.0
[    0.000000] percpu: Embedded 17 pages/cpu s38924 r8192 d22516 u69632
[    0.000000] pcpu-alloc: s38924 r8192 d22516 u69632 alloc=17*4096
[    0.000000] pcpu-alloc: [0] 0 [0] 1 [0] 2 [0] 3
[    0.000000] Built 1 zonelists, mobility grouping on.  Total pages: 256832
[    0.000000] Dentry cache hash table entries: 131072 (order: 7, 524288 bytes)
[    0.000000] Inode-cache hash table entries: 65536 (order: 6, 262144 bytes)
[    0.000000] mem auto-init: stack:off, heap alloc:off, heap free:off
[    0.000000] Memory: 960940K/1034240K available (11264K kernel code, 985K
rwdata, 3180K rodata, 1024K init, 2104K bss, 32340K reserved, 40960K cma-
reserved, 245024K highmem)
...
```

First CMA allocation success message

```
[    0.000000] Reserved memory: created CMA memory pool at 0x88000000, size 24 MiB
[    0.000000] OF: reserved mem: initialized node region@88000000, compatible id shared-dma-pool
```

Second CMA allocation success message (system default)

```
[    0.000000] cma: Reserved 16 MiB at 0x9f000000
```